Threads 1 (Ejemplos de uso de hilos)

El código incluye diferentes ejemplos en varios paquetes:

- Crear, dormir, interrumpir, pausar y esperar a que termite un Thread.
- Ejecución secuencial y concurrente.
- Acceso concurrente a datos.
- Ejemplos básicos de Swing.

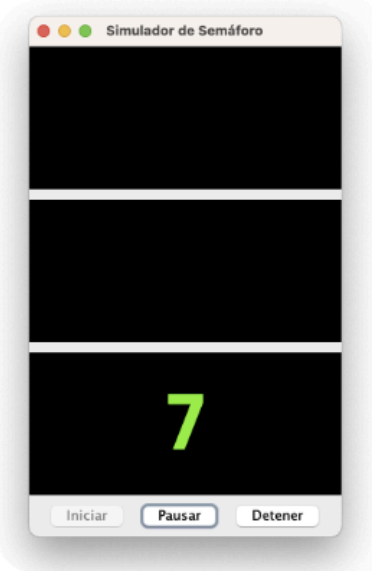
Threads 2 (progress bar)

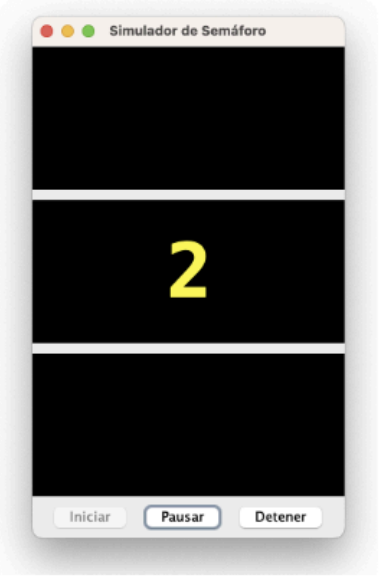
Crea una clase que haga lo siguiente:

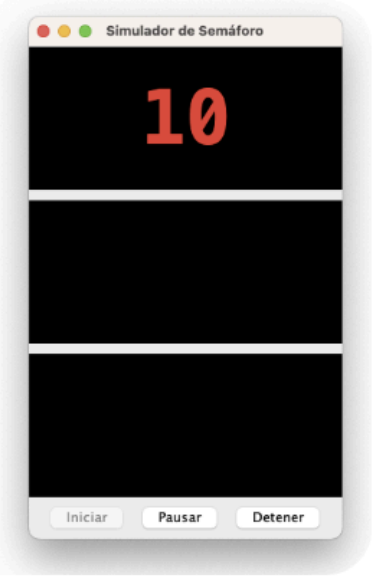
- Crea una ventana con dos botones: “Iniciar” y “Parar”, que estarán habilitado y deshabilitado respectivamente.
- Añade un JProgressBar a la parte inferior (SOUTH) de la ventana configurada para mostrar valores entre 0 y 100 así como el texto del % de progreso sobre ella.
- Cuando se pulse “Iniciar” lanza el thread que cuente hasta 10_000_000 (imprime los valores en la consola).
- Haz que el thread actualice la posición de la barra de progreso en el porcentaje que corresponda al estado actual del contador.
- Cuando el thread termine restablece los botones para poder volver a iniciar y por a 0 la barra de progreso.
- Añade la funcionalidad para detener correctamente el thread si se pulsa “Parar”.

Threads 3 (semáforo)

Vamos a crear una aplicación que simule el funcionamiento de un semáforo. El semáforo debe cambiar de color automáticamente siguiendo un ciclo definido y permitir pausar y reanudar el ciclo en cualquier momento. Además, debe mostrar en cada luz del semáforo una cuenta regresiva con el tiempo restante para el cambio de fase (Verde, Amarillo, Rojo). Para resolver esta práctica debes tener en cuenta lo siguiente:







Representación visual del semáforo

- Crea 3 labels que representen las luces del semáforo: Rojo, Amarillo y Verde.
- Las etiquetas deben estar dispuestas verticalmente, simulando la apariencia de un semáforo real (Rojo, Amarillo, Verde).
- Cada etiqueta debe mostrar el tiempo restante para el cambio de luz durante su fase activa usando el color de la luz correspondiente.

Botones de control

- **Iniciar**: Comienza el ciclo del semáforo. Este botón debe estar habilitado al inicio y deshabilitarse una vez que el ciclo ha comenzado.
- **Pausar/Reanudar**: Permite pausar y reanudar el ciclo del semáforo. El texto del botón debe cambiar dinámicamente entre "Pausar" y "Reanudar" según el estado actual.
- **Detener**: Detiene completamente el ciclo del semáforo y reinicia la aplicación a su estado inicial. Debe habilitarse una vez que el ciclo ha comenzado.

Lógica del semáforo

- El semáforo debe seguir el siguiente ciclo de fases en bucle continuo:
    - Verde: 10 segundos.
    - Amarillo: 3 segundos.
    - Rojo: 10 segundos.
- Cuenta regresiva:
    - Durante cada fase, la etiqueta de la fase activa debe mostrar una cuenta atrás del tiempo restante en segundos (se hacen pausas de 1seg. entre cada número).
    - Al finalizar cada fase, la cuenta atrás debe reiniciarse para la siguiente fase, y la etiqueta de la fase anterior debe vaciarse (no mostrar texto).
- Reinicio de ciclo:
    - El ciclo debe repetirse indefinidamente hasta que se detenga la aplicación o se pulse el botón "Detener".

Gestión de hilos: Implementa un hilo separado para gestionar el ciclo del semáforo y la cuenta regresiva, evitando bloquear la interfaz gráfica. El hilo debe manejar el cambio de fases y actualizar el tiempo restante en las etiquetas correspondientes.

- **Control de pausa y reanudación**
    - Implementa la funcionalidad de pausar y reanudar el hilo del semáforo cuando se pulse el botón "Pausar/Reanudar".
    - Utiliza mecanismos de sincronización adecuados (wait() y notifyAll()) para controlar la pausa y reanudación del hilo.
- **Detención del hilo**
    - Al pulsar el botón "Detener", interrumpe el hilo del semáforo de forma segura utilizando interrupt().
    - Utiliza join() para esperar a que el hilo termine antes de reiniciar la aplicación.
    - Asegura que no queden hilos en ejecución al detener la aplicación.